Agentic AI on AWS Fundamentals

Dai Truong

Senior Partner Solutions Architect

Discussion Points

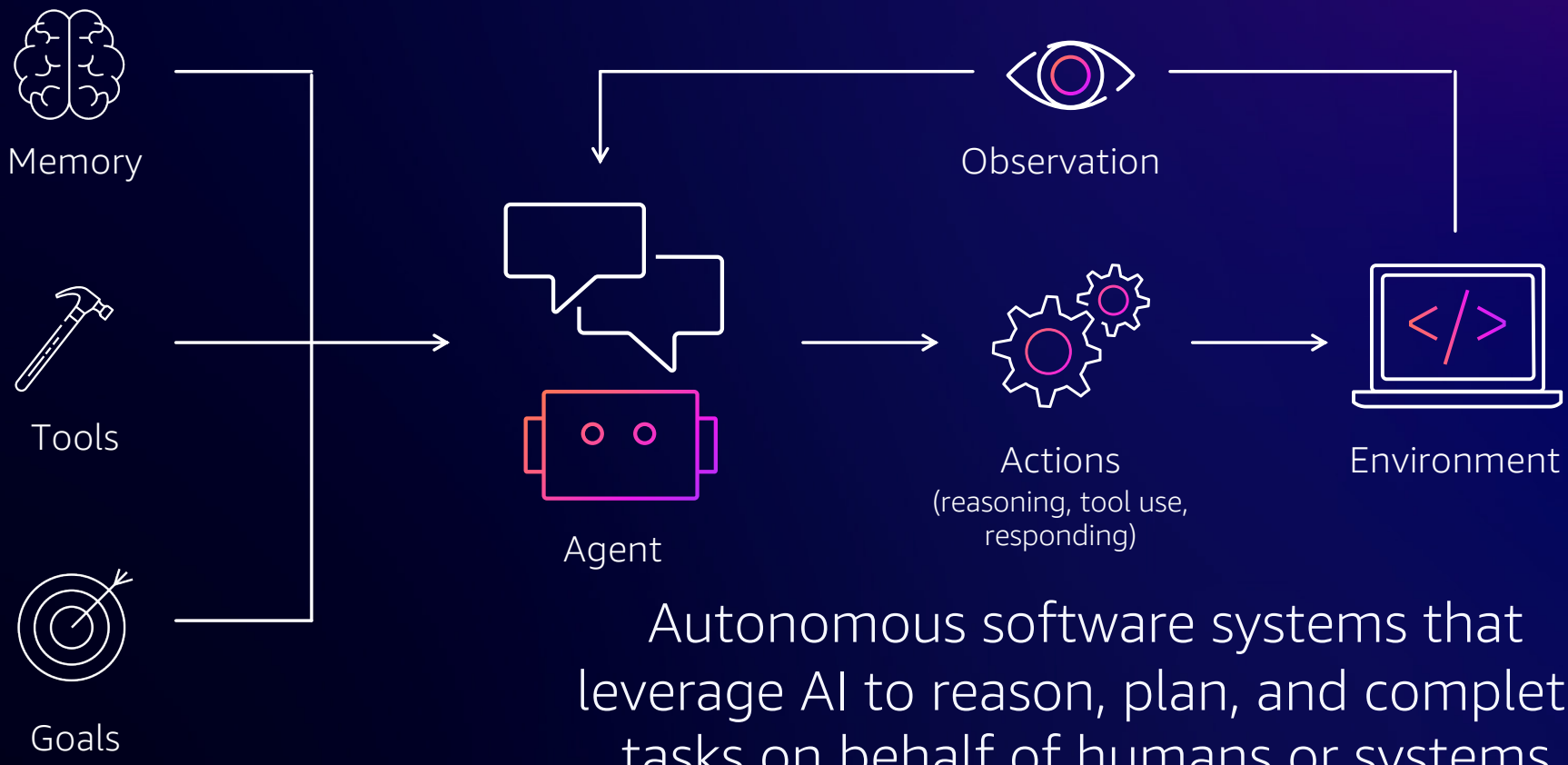
- What is Agentic AI?
- When to apply Agentic AI?
- Key Considerations for implementing Agentic AI
- Industry PoVs for Agentic AI
- What are the Agent Patterns and Protocols?
- Amazon Bedrock AgentCore

What is Agentic AI?

What are AI Agents?

Autonomous or semi-autonomous software systems that can reason, plan, and act to accomplish goals in digital or physical environments.

What is Agentic AI?



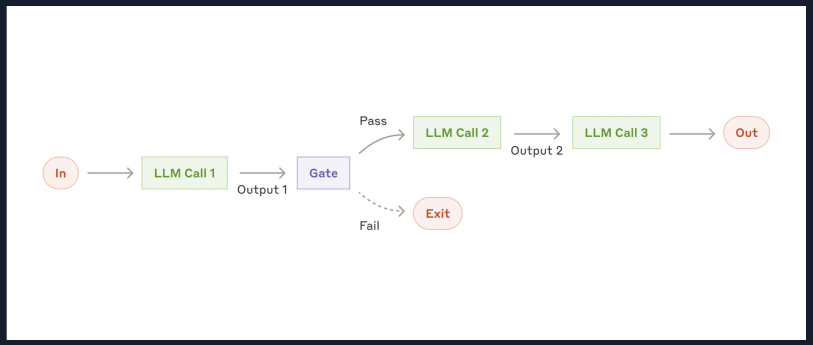
Autonomous software systems that leverage AI to reason, plan, and complete tasks on behalf of humans or systems

Workflows vs Agents

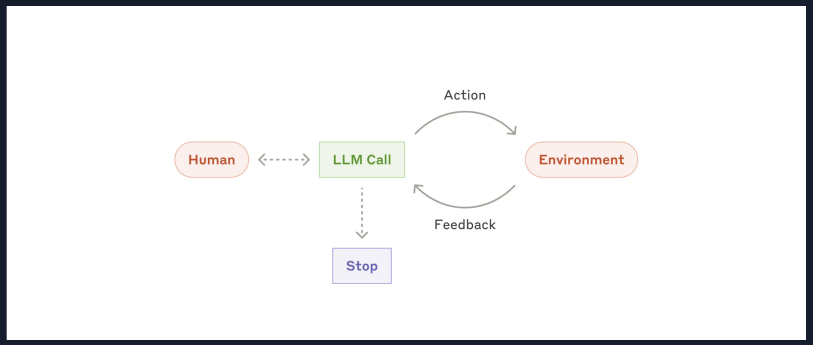
- Workflows operate through pre-defined code paths
  - Examples: Customer service routing to different departments and specialized response generation, etc.

- Agents dynamically chooses processes and tools to use
  - Examples: customer support agents, software devepment agents, market research analysts, etc.

Workflow



Agent



Industry PoVs for Agentic AI

Generative AI use cases across industries

Enhance Customer Experiences

- Chatbots
- Virtual assistants
- AI-powered contact center
- Conversation
- Analytics
- Personalization

Boost Employee Productivity & Creativity

- Conversational search
- Summarization
- Content creation: writing, media, design, modeling
- Code generation
- Data to insights

Optimize Business Processes

- Document processing
- Data augmentation
- Cybersecurity
- Fraud detection
- Process optimization
- Agentic AI workflow acceleration

When to apply Agentic AI?

Choose between workflows vs agents?

When tasks are well-defined, use workflows. Well-designed workflows have benefits of more controlled behaviors, lower costs and latency.

When tasks are open-ended, processes can’t be pre-determined, use agents; However, agents behavior can be quite underministic.

An Agent's Toolbox

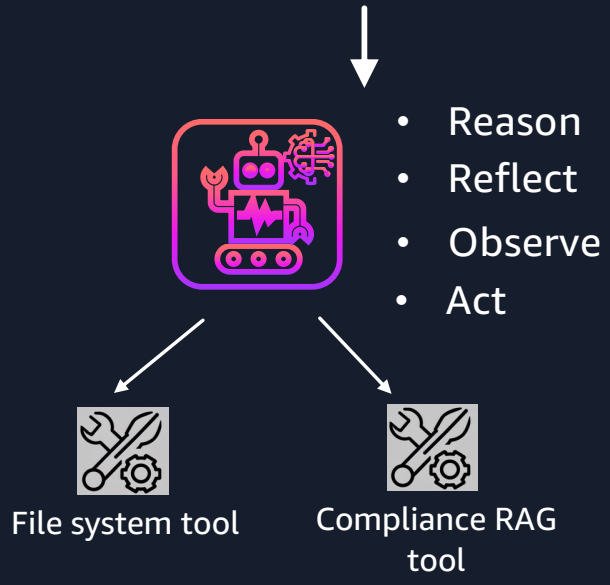
Some of the most common tools that developers provide to agents include:

- Web search
- Semantic retrieval (RAG)
- Structured retrieval (text-to-query language)
- Code interpreter
- File system operations
- Shell integration
- Browser-use
- Long-term memory
- Other models (e.g. Computer Vision models)
- Other agents!

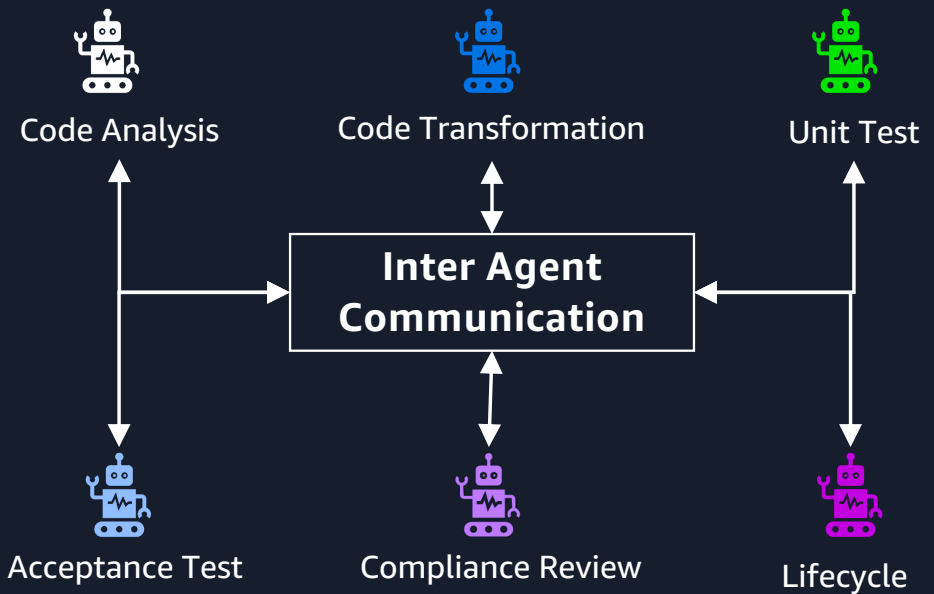
Single Agent Vs Multi-Agent Systems

SCALING THE COMPLEXITY

"Can you check this code according to our compliance standard?"



"Can you refactor this project to meet new compliance guideline?"



Key Considerations for implementing Agentic AI

Scale requires operational excellence

Excitement and potential

POC

Challenges on the path to production

Performance

Scalability

Security

Context

Governance

Meaningful business value

AI production agents

Things to keep in mind when building agentic solutions

- Context is king - context engineering (key details, events, and decisions) is critical.
- Reliability through simplicity. Start small, scale smart.
- For many use cases, single agent is sufficient.
- SLMs (small language models via model distillation) for context compression.
- Enable ample observability.
- Always think about tradeoffs between performance, cost and latency.

Choosing Between Multi-Agent and Single-Agent Systems: A Context Sharing Framework

Single-Threaded Agents excel when:

- Tasks require heavy coordination between steps
- Changes in one area impact the whole
- Unified context is critical

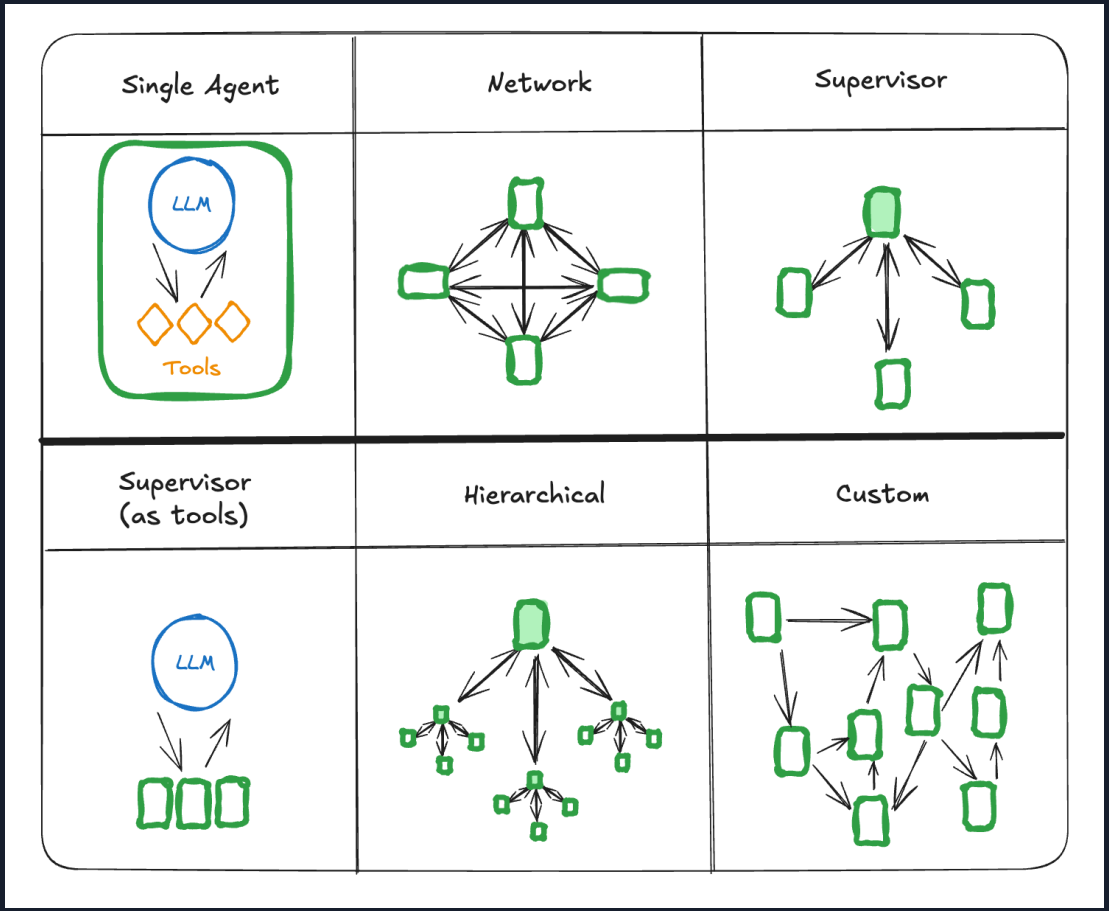
Example: Coding assistants like Claude Code — parallel agents making independent code changes would create conflicts and break dependencies

Multi-Agent Systems shine when:

- Tasks can be parallelized with minimal overlap, e.g. breadth-first queries that involve pursuing multiple independent directions simultaneously
- Sub-tasks benefit from isolated contexts
- Independent processing speeds completion

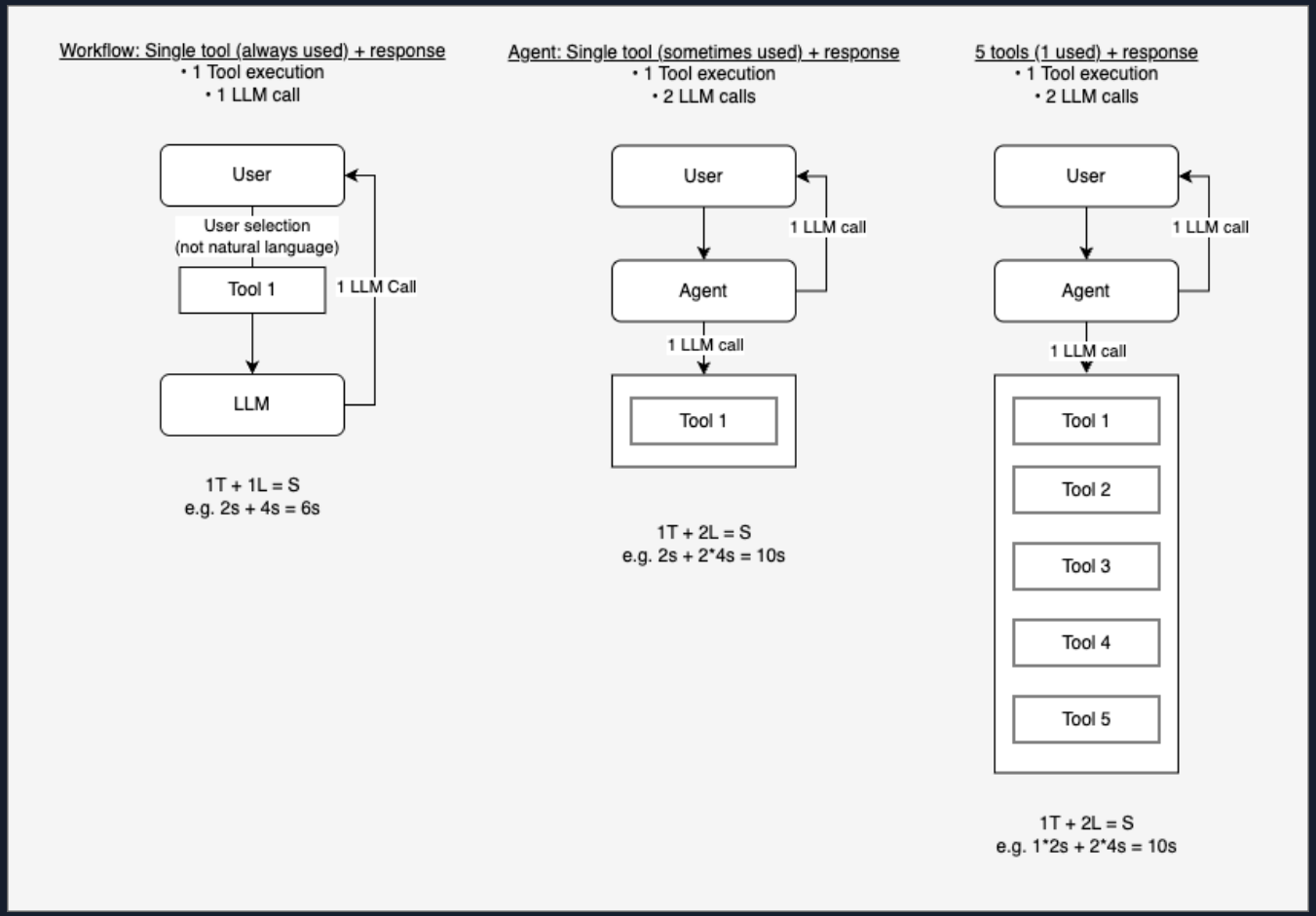
Example: Deep research assistants — multiple agents can explore different sources simultaneously without interference

Multi-agent Architectures

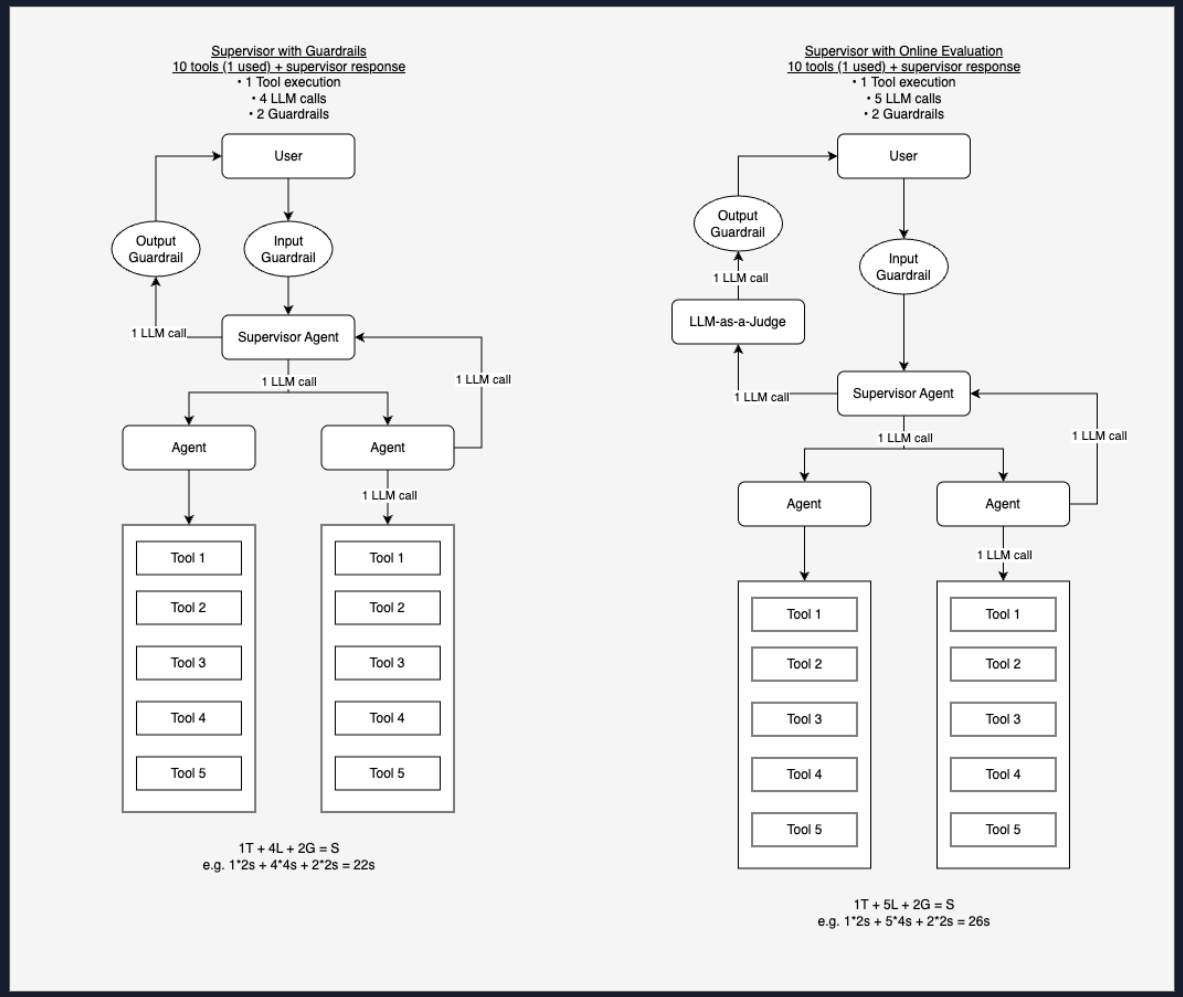


https://langchain-ai.github.io/langgraph/concepts/multi_agent/

Workflow vs. Agent Latency Patterns



Production System Latency Patterns



Agent Resources

- Building Effective Agents
  - https://www.anthropic.com/engineering/building-effective-agents
- 12 factor agents
  - https://github.com/humanlayer/12-factor-agents
  - https://www.youtube.com/watch?v=8kMaTybvDUw
- Single Agents vs Multi-agent:
  - https://www.anthropic.com/engineering/built-multi-agent-research-system
  - https://cognition.ai/blog/dont-build-multi-agents
- Andrei Karpathy – Software is Changing (again)
  - https://www.youtube.com/watch?v=LCEmiRjPEtQ

Why Agent Development Initiatives Often Fail

Planning & Scoping Issues

- Unrealistic or vague scope
- Unrealistic accuracy/latency/cost targets
- Misaligned use case (e.g. deterministic workflow more suited)

Technical Architecture Problems

- Poor choice of agent coordination architecture (e.g. multi-agent)
- Poor context engineering
- Lack of framework understanding (e.g. overuse of high-level abstractions)

Development Process Failures

- Lack of evaluation pipelines and observability
- No evaluation = no methodical development progress
- Inadequate testing

Accuracy, Cost, Latency

Reality Check

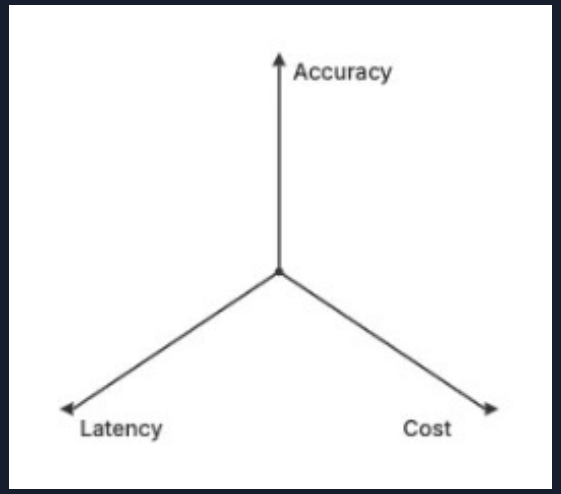
- High Accuracy + Low Cost + Low Latency = Usually impossible
- Pick two, compromise on the third
- Most teams are unrealistic about all three

Common Mistakes

- Expecting Claude Opus quality at Nova Lite cost
- Wanting real-time responses with complex workflows
- Assuming you can optimize all three simultaneously
- Assuming you can get all three with short development timelines

Current Technology Limitations

- Quality models are expensive and slow
- Fast/cheap models lack reliability
- No magic solutions exist



Performance Quality is Still the Biggest Blocker

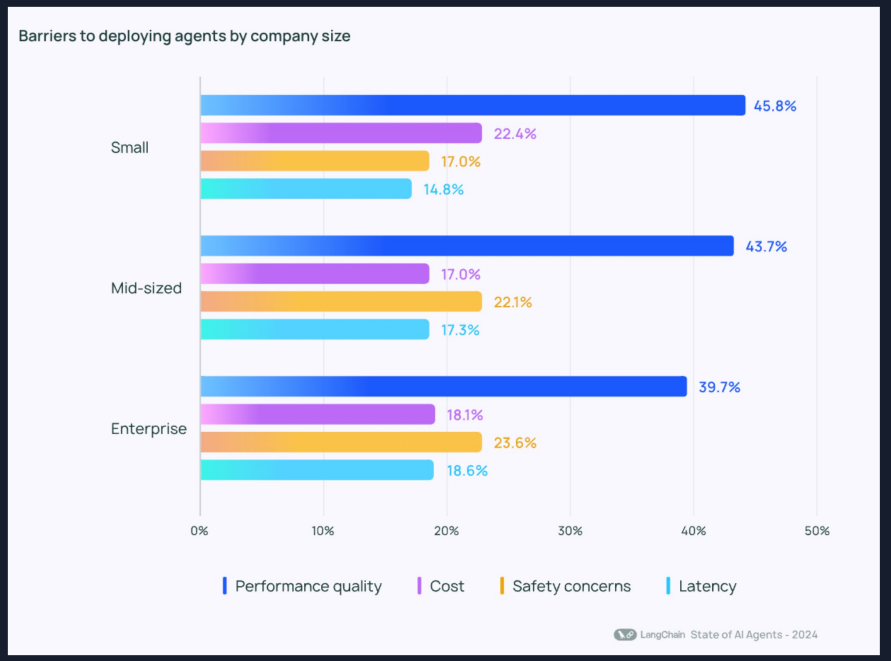
According to LangChain's State of AI Agents Report, performance quality remains the #1 challenge for production agents.

The Hard Truth:

- It's not about deployment infrastructure.
- It's not about scaling challenges.

It's simply: The agents don’t work well enough.

If there's a compelling enough reason to get an agentic system to production from a performance perspective, developers will in most cases find a way to get over the deployment hurdles.



https://www.langchain.com/stateofaiagents

Premature Optimization is the Root of All Evil

The Classic Mistake: **Teams try to optimize cost and latency before achieving accuracy**

Why This Fails

1. You can't optimize what doesn't work
2. Accuracy is exponentially harder than expected
3. Optimization often hurts quality
4. You'll likely need to start over anyway

The Right Approach

1. Get it working (ignore cost/latency)
2. Make it reliable (still ignore cost/latency)
3. Optimize if needed

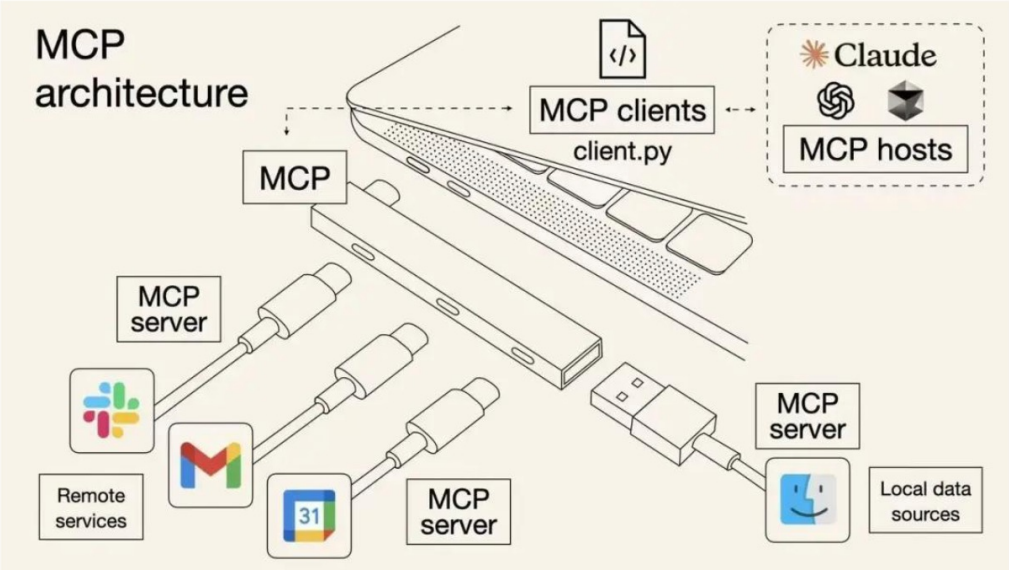
Aside: Once it actually works well, users often accept higher costs/latency than you expected (especially when you design thoughtful UX around agentic experiences).

&

We Need a reliable Agent Evaluation Framework:

- Trajectory comparison tools
- Tool usage analytics
- Conversational / multi-step success metrics
- Statistical significance testing for agents

MCP in simple terms – USB for AI



- MCP helps reduced complexity
- MCP decouples AI development from specific backend system
  - MCP client focuses on LLM, agent, application layer focused
  - MCP server focuses on data sources connectivity
- Hence, if you have new data sources, just connect to MCP server to make discoverable by MCP client (hence, AI agent)

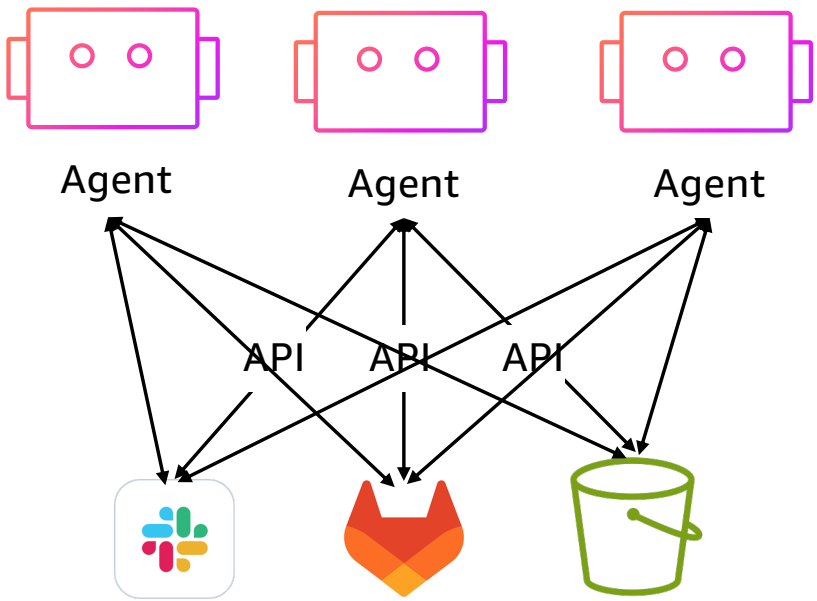
* for business users analogy only, as this is not accurated from technical perspective

Standardization, integration, flexibility, scalability

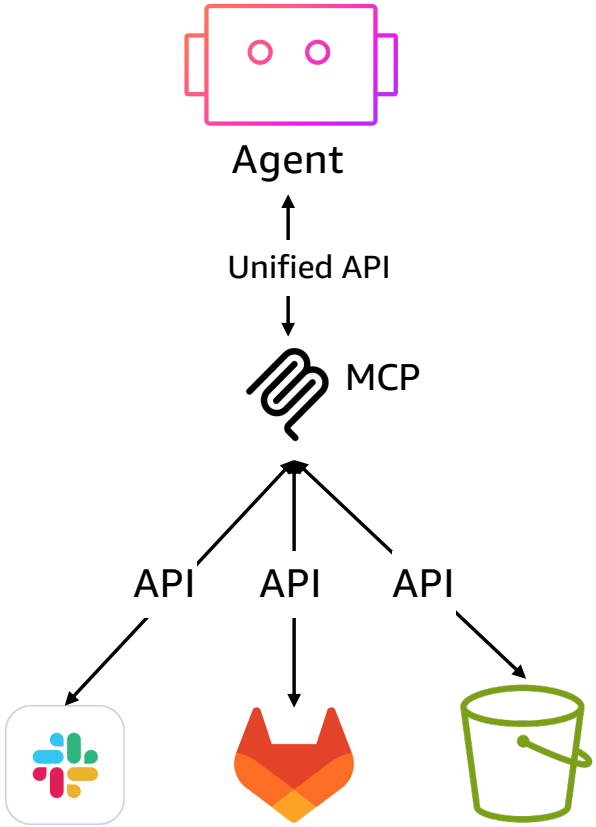
Agentic Interoperability

MCP: Simplifying Communications with Tools, Resources

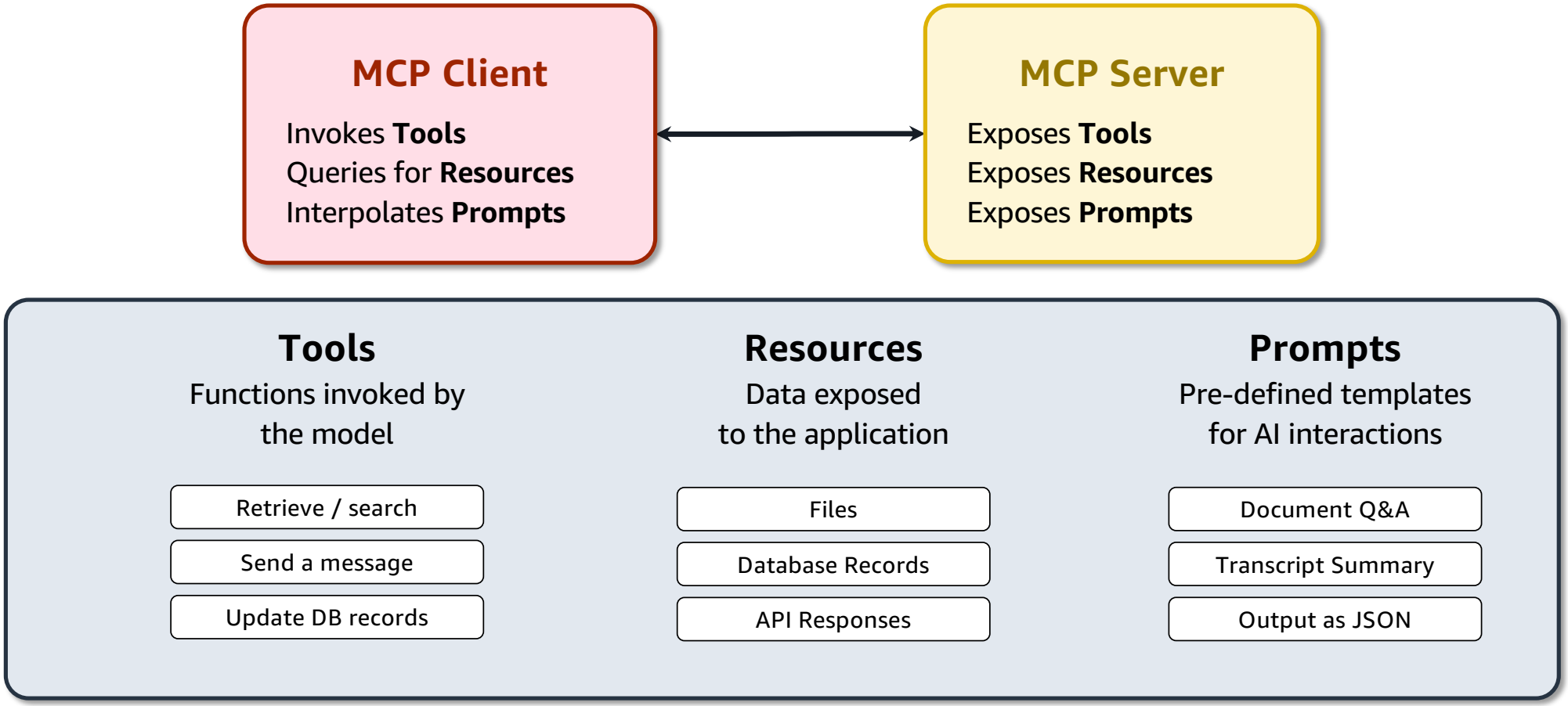
Before MCP



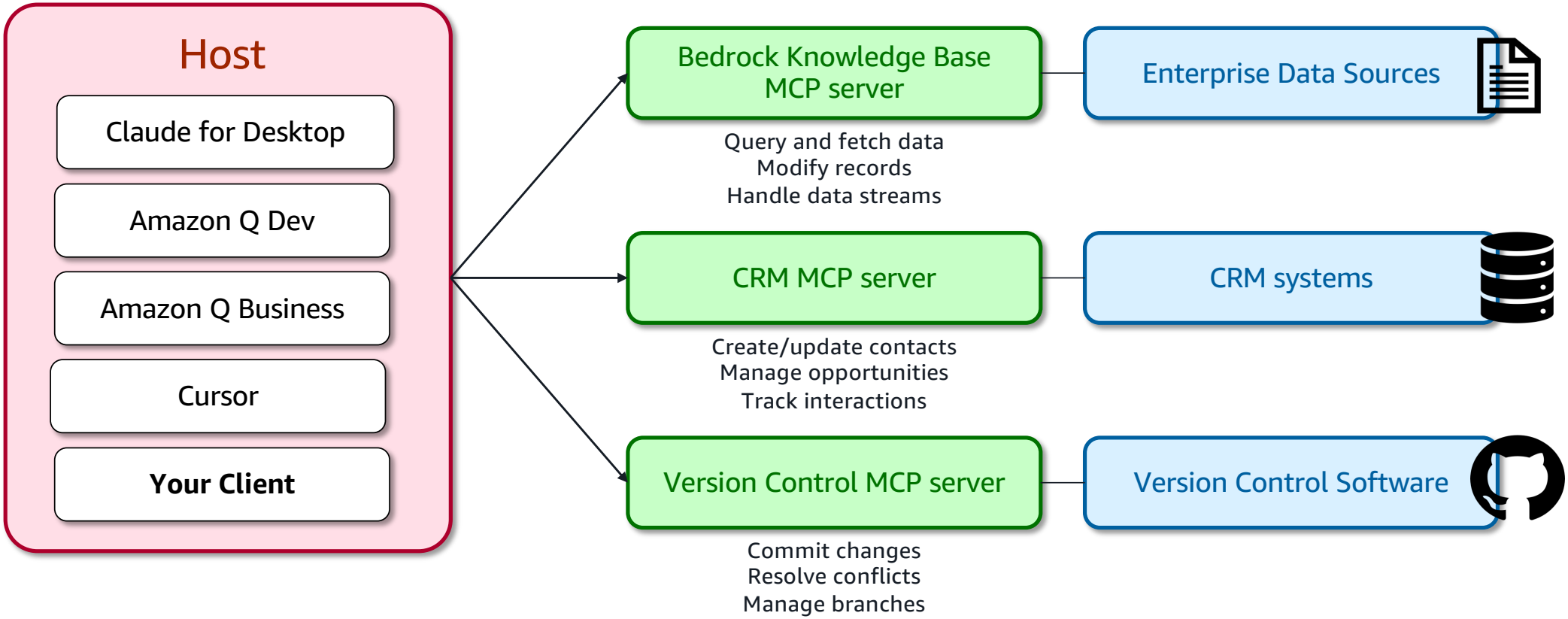
After MCP



MCP Deep-Dive



With MCP: Standardized AI Development



1. **Host:** A program, AI tool, or AI agents requiring access to data through MCP
2. **MCP Client:** Protocol clients maintaining one-to-one connections with servers
3. **MCP Server:** Lightweight programs exposing capabilities through standard MCP
4. **Data sources:** both local (databases, files) and remote services (APIs) that MCP server can access

Dive deeper on MCP

https://modelcontextprotocol.io/introduction

https://github.com/modelcontextprotocol/

https://github.com/aws-samples/sample-serverless-mcp-servers

https://github.com/awslabs/mcp

A2A Protocol – Complementing MCP

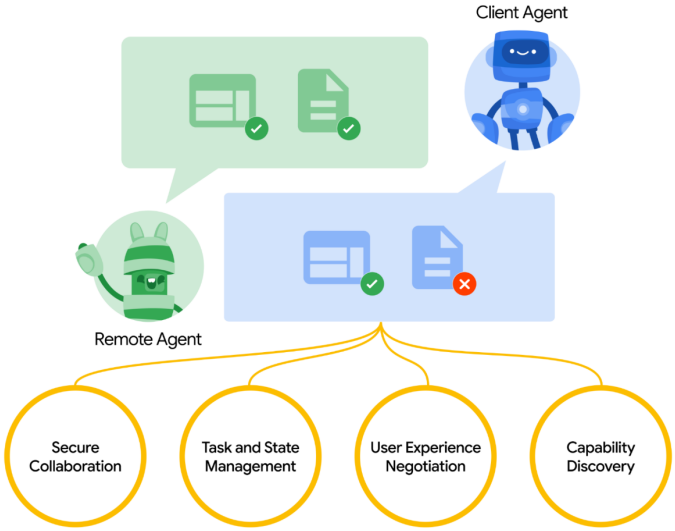
Google released Agent2Agent (A2A) Protocol as an open standard for agents (client and remote agents) to communicate with each other. Key capabilities are:

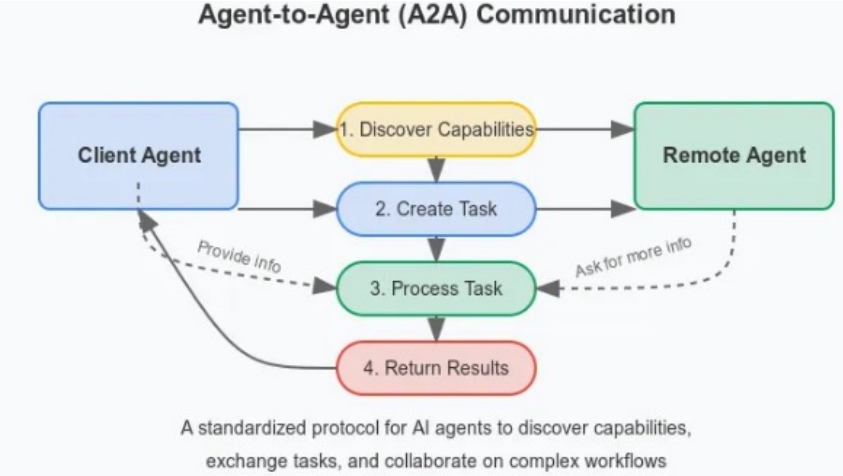
Capability discovery: Agents can advertise their capabilities using an **"Agent Card"** in JSON format, allowing the client agent to identify the best agent that can perform a task and leverage A2A to communicate with the remote agent.

Task management: Client agent is responsible for formulating and communicating "**tasks"**, while the remote agent is responsible for acting on those tasks.

Collaboration: Agents can send each other messages to communicate context, replies, artifacts, or user instructions (long running tasks).

Negotiate interaction modalities: Client and remote agents negotiate the message format based on the user's UI capabilities e.g., text, iframes, web forms, and media.





MCP + A2A Agentic System: Retail Example

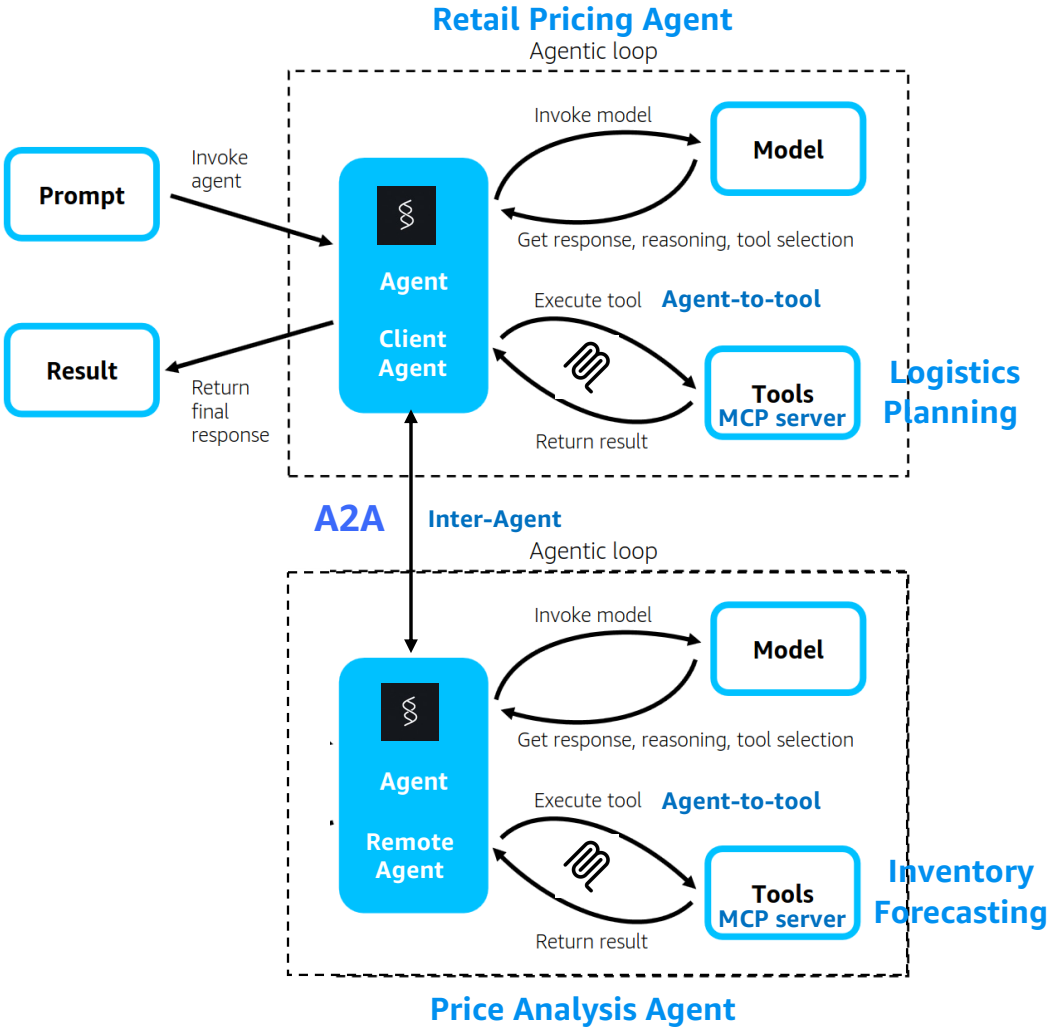
Retail pricing agent (Orchestrator):
Receives natural language requests from store managers and delegates to specialized agents/tools to answer the question.

Specialized tools as MCP servers

- Logistics planning
- Inventory forecasting
- Processes data across AWS services, eg: S3, DynamoDB, Lambda

Specialized agent access via A2A

- Price analysis agent



Protocol Characteristics

	MCP (Model Context Protocol)	A2A (Agent2Agent Protocol)
Primary Purpose	Connect AI models to external tools and systems	Enable communication between AI agents
Steward of standard	Anthropic introduced MCP in Nov'24. AWS is on MCP steering committee.	Google introduced A2A in Apr'25
Agent Interoperability focus	Agent-to-tool/resource AWS recommended pattern: Inter-agent communication	Agent-to-agent collaboration
Architecture	Client-server with stateful connections. Request-response for tool execution.	Peer-to-peer with task-based workflows
Transport Layer	Uses JSON-RPC 2.0 over STDIO, Server-Sent Events (SSE) for remote streaming, and new Streamable HTTP for bidirectional messaging through a single endpoint	JSON-RPC 2.0 over HTTP(S) for standard communication, plus Server-Sent Events (SSE) for real-time task progress streaming and push notifications via webhook URLs
Security	Uses OAuth 2.0/2.1 based authentication and authorization to verify agent identities and manage access controls. End-to-end encryption to protect data during transmission and at rest.	Uses OAuth 2.0 based authentication and authorization mechanisms. Implements RBAC to manage agent permissions. End-to-end encryption to protect data exchange.
State Management	Context-aware session management across API calls. Maintains state between client and server with sessions IDs.	Task lifecycle management with states (submitted → working → completed). Uses asynchronous state tracking through Server-Sent Events (SSE) for long-running tasks.
Discovery Method	MCP client queries the server to discover available tools, resources, and prompts.	A2A uses a standardized Agent Card system designed for peer-to-peer agent discovery and collaboration. AgentCard at path: /.well-known/agent.json
Agent use-case example	Sales agent accessing CRM database over MCP. Sales agent communicating with Customer Service agent over MCP inter-agent.	Sales agent accessing CRM database over MCP. Sales agent collaborating with Customer Service agent over A2A.

What are Agent Frameworks?

Goals

Agentic AI Framework: A development framework that supports built-in features for building agent so takes away some heavy-lifting (in theory)

What a Framework needs to do

- Tool integration and orchestration (w/ MCP support)
- Plug-and-play for range of models and data sources
- Memory and state management
- Agent planning and decision making
- Multi agent collaboration
- Easy to use
- Observability and debugging
- Production ready (concurrency/async)

Make sure you are adding value, not adding technical debt

(if you used LangChain, you may be buried under a mountain of abstraction)

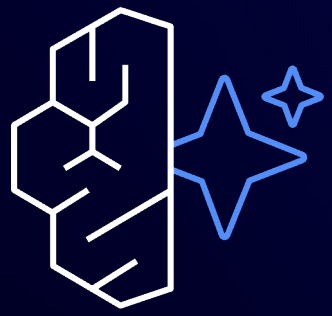
Develop solutions that stand the test of time - *Ride the wave* of GenAI

~~Invent~~ Innovate and ***Simplify***

Some agent frameworks…

	LangGraph	CrewAI	Strands
Approach	Graph based workflows / state machines – very customizable; Functional API	Declarative – role based crews and tasks with flows	Modular composable 'strands'/agents – also support graphs
Complexity	High (steep learning curve)	Medium	Low-medium
Community	15.3k stars	33.9k stars	1.9k stars
Tool integration	Strong – tight with LangChain	Good – built-in and custom tools	Good – many built-in and custom tools
Multi-agent	Some support	First class support	Support with A2A, Agents-as-tools
Production readiness	Medium-High (LangChain ecosystem)	Medium	Medium
Language support	Python, JS	Python	Python
Enterprise support			

Amazon Bedrock AgentCore



Amazon Bedrock AgentCore

Deploy and operate highly capable agents securely, at scale using any framework and model

- Purpose built Agent Runtime and Identity
- Simplified tools discovery and integration
- Short-term and long-term memory
- End-to-end visibility to trace, debug and monitor agents
- Fully managed browser tool and code interpretation capabilities

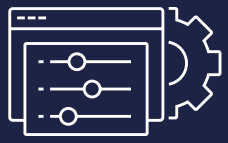
AgentCore Runtime

Framework and model independent deployment

- Support for Strands Agents, LangChain, LangGraph, CrewAI and other agentic frameworks
- Use Amazon Bedrock, Amazon SageMaker, OpenAI, Gemini or any other model on your agent

Use case independent

- Handle large payload sizes: text, images, audios, videos and others
- Fast initialization and long running asynchronous workloads
- Host agents and tools

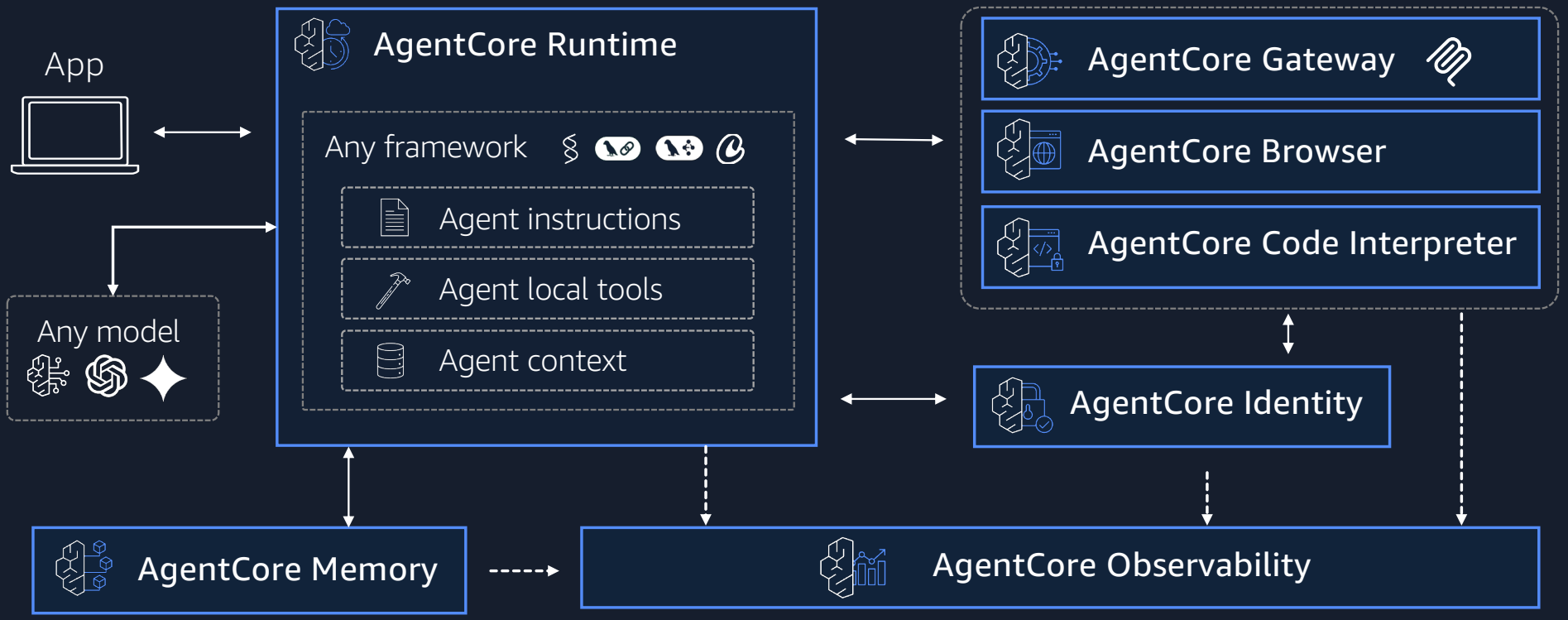


Secure workload with enterprise-grade isolation

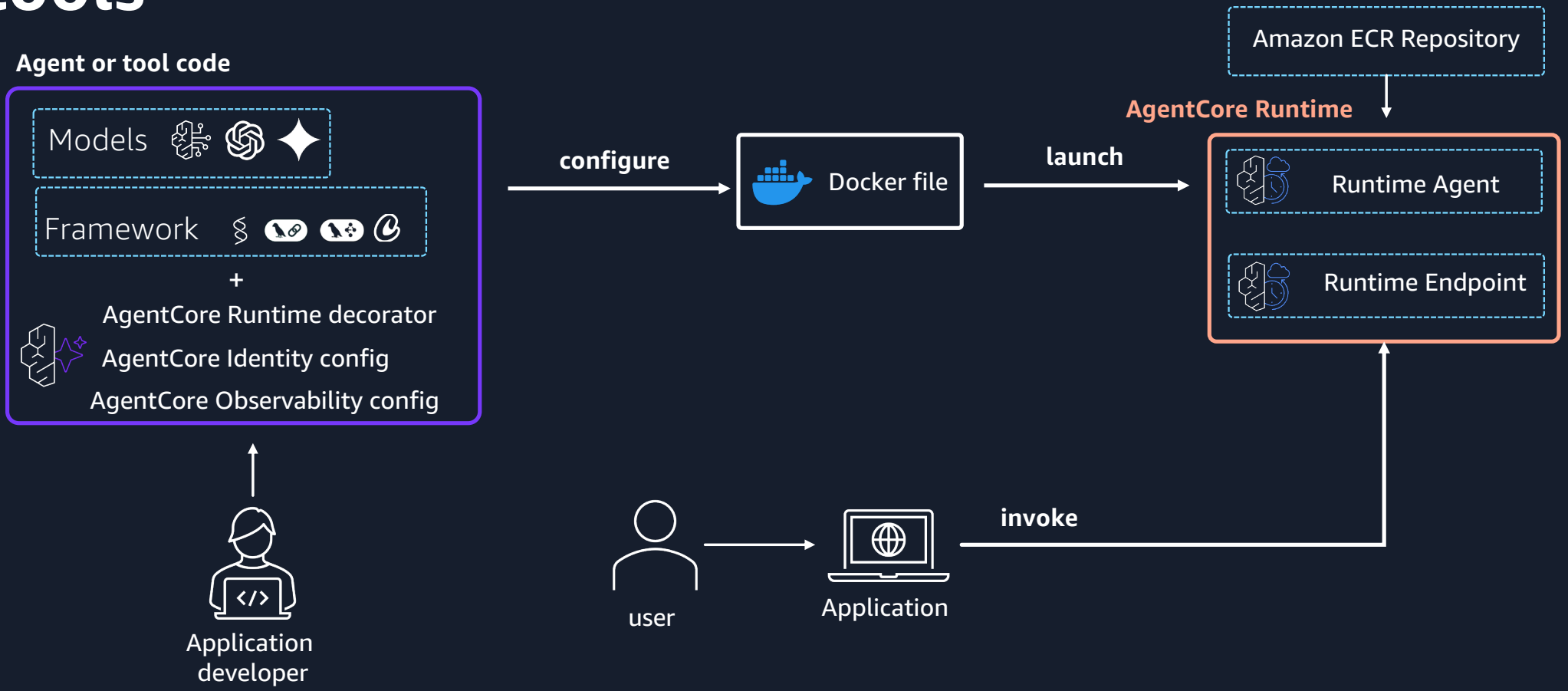
- True session isolation with persistent dedicated execution environments
- Built-in authentication and observability capabilities

AgentCore for production-ready agents

ANY MODEL, ANY AGENT FRAMEWORK



Secure and scalable runtime for agents and tools



AgentCore Runtime – what will customers love?

Any framework, any model	Supporting sync and async agents	Response streaming	Large payloads (100MB)
Long running agents (up to 8 hours)	Fast startup (200ms)	Hosting MCP tools	Session isolation
Only 4 lines of code to deploy existing agent	Session id's for context handling	Multi-modality support	Getting started SDK

AgentCore Gateway

Simplify tool development and integration

- Transform APIs into agent-ready tools without custom code or infrastructure
- Supports RESTful services via OpenAPI schema, and AWS Lambda functions

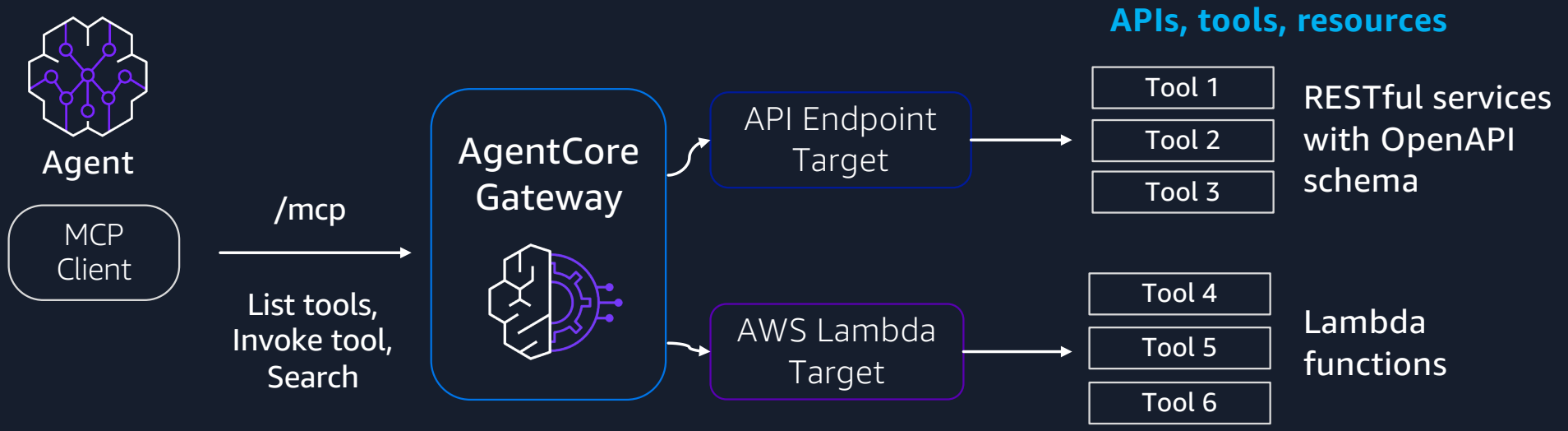
Secure and unified tool access

- Simplified cross-organizational tool sharing with enterprise-grade security
- Built-in inbound and outbound authentication and observability

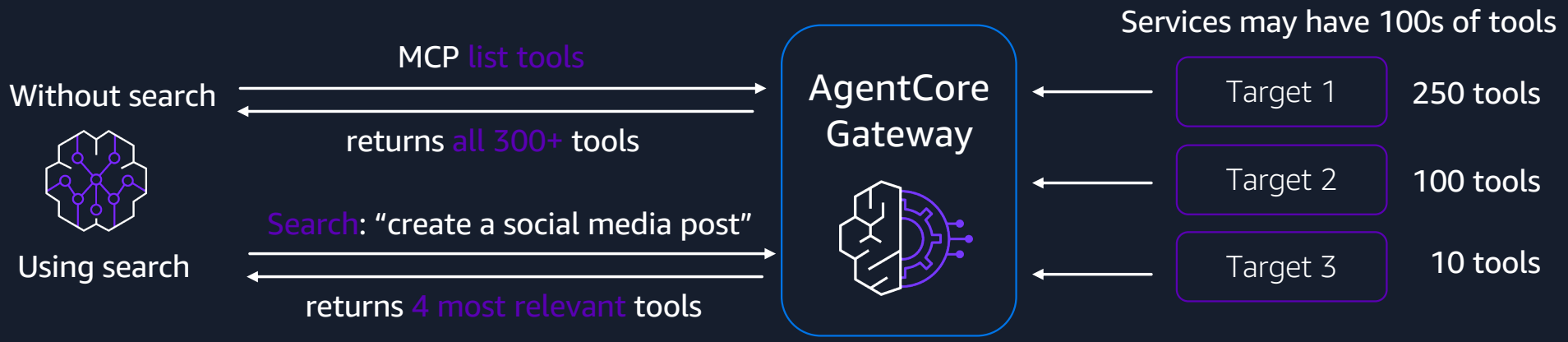
Intelligent tool discovery capabilities

- Expose tools using MCP enabling tool discoverability
- Built-in semantic search matches tools to agent tasks

Unified, secure, agent-ready tools



AgentCore Gateway semantic search



Benefits

- AgentCore Gateway automatically indexes tools, and gives serverless semantic search
- Reduces context passed to the agent’s LLM, improving accuracy, speed, and cost
- Lets agent focus on tools relevant for a given task

AgentCore Memory

Short-term and long term memory management

- Long term memory capabilities from session interactions
- Handle multiple memory strategies for a single conversation

Fully managed and secure

- Abstracts memory infrastructure
- Built-in encryption
- Flexible namespace for memory sharing

Retrieval and extraction of memory

- Multiple memory retrieval patterns – semantic search, filter by namespace
- Built-in and custom memory extraction strategies – summary per session, user preferences, semantic memory

AgentCore Identity

Secure access to agents and tools

- Distinct identities for secure agent operations at scale
- Authentication with enterprise identity providers
- Secure credential management for external service access and integration

Minimized consent fatigue

- Reduces need for repeated authorization
- Streamlines authentication flows
- Simplifies user experiencer for all agent-powered interactions

Accelerated AI agent development

- Preserves existing identity systems such as Okta, Microsoft Entra ID, or Amazon Cognito
- Inbound and outbound authentication

AgentCore Observability

Full transparency of agent behavior

- Visualize agent workflows with detailed traces in deep dive dashboard
- Comprehensive monitoring for latency, tokens, tools and custom metadata

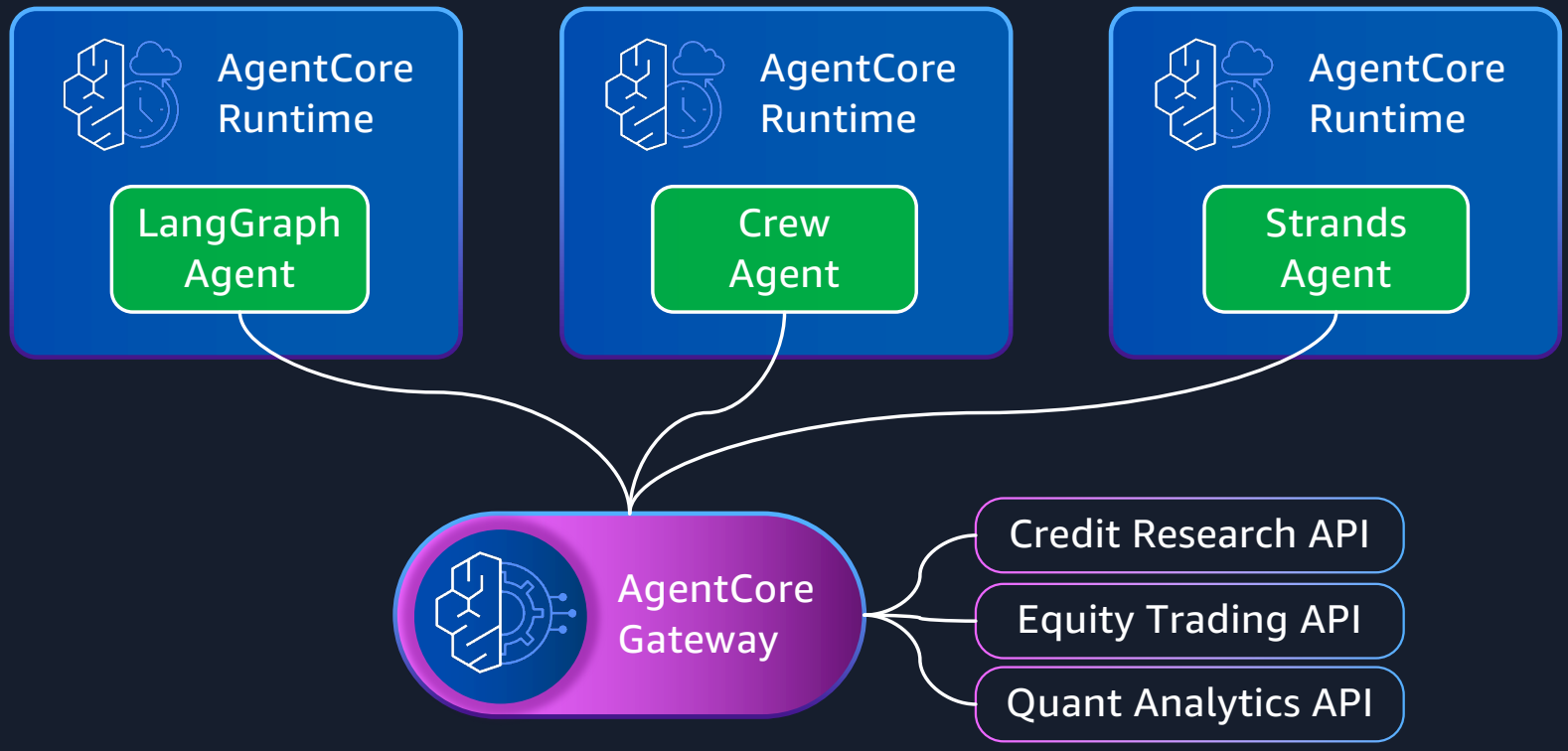
Enterprise ready security and governance

- Enterprise ready with IAM access controls and PII redaction capabilities

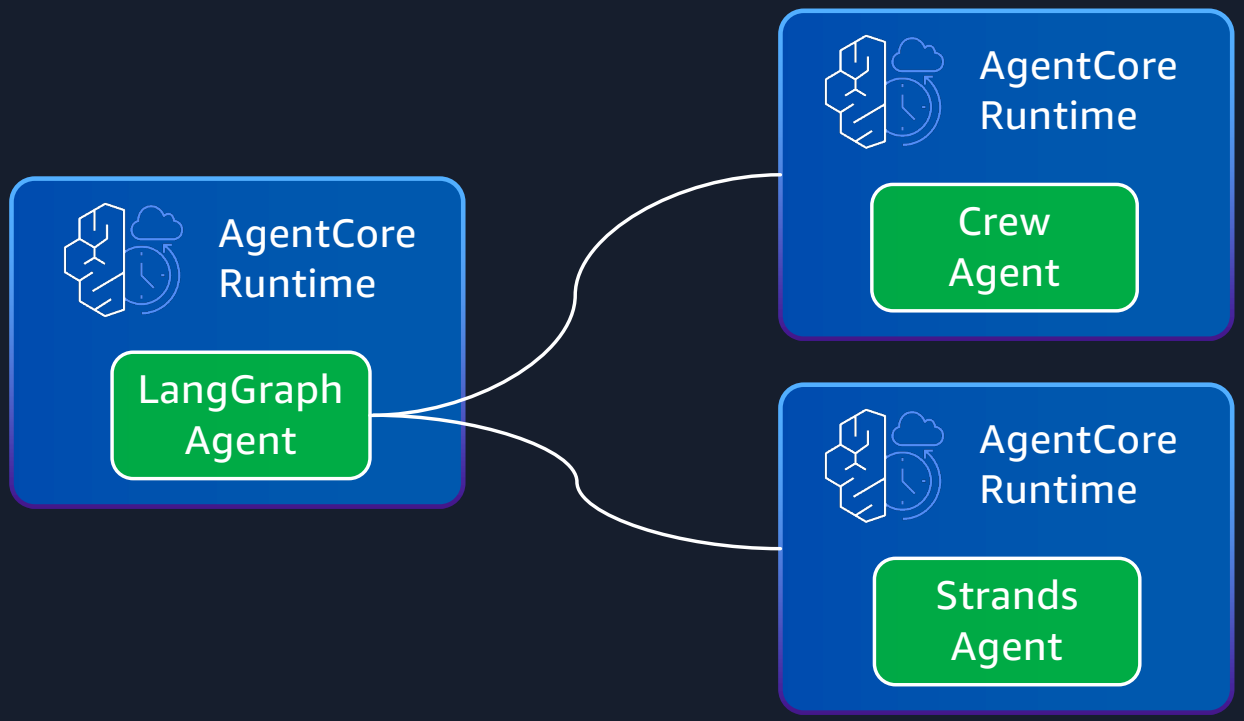
Integrate with 3P Observability tools

- OpenTelemetry (OTEL) compatible for integrating logs, metrics, and traces
- Flexibility to leverage your existing observability stack

Reusable curated MCP tools accessed by agents using different frameworks, from Runtime



Multi-agent collaboration with supervisor and sub-agents using different frameworks, all on Runtime



Thank you!